

Deploying Self-Regulating Cars

Anonymous Authors¹

Abstract—A free-flow road network jams when a local disturbance pushes density past a critical point. Earlier work learned a controller that prevents this by setting a desired speed per road segment, and measured a throughput gain for it in simulation. That controller reads vehicle-level state a central observer collects, and it delivers its decisions through a cloud service. This paper asks what has to be added to an ordinary vehicle for the same controller to run on the vehicle instead, and reports what happened when that was built and driven.

The answer has four parts: an Android phone the driver already owns, an NVIDIA Jetson Orin Nano developer kit listing at about US\$250, a public traffic API, and a filter that bounds the advisory before it is shown. Nothing is installed on the road, and no server sits inside the control loop. The policy is a deterministic function of a feed every vehicle can query, so vehicles carrying the same weights compute the same action without coordinating with each other. The phone samples its camera, GPS and inertial unit at rates the Jetson sets at run time rather than at install time.

Eight drives on one day recorded 22,929 ticks over 152.8 minutes on vehicle power. The on-Jetson path has a median of 19.8ms, and the phone-to-Jetson path has a 95th percentile of 116.19ms against a 200ms target. Nothing was connected to the vehicle’s controls, and no advisory was acted on by a driver. In simulation on the same network, the vehicle-side observation gains 230 ± 44 veh/h over no control and the published observation gains 224 ± 61 veh/h, so restricting the observation to what a feed returns costs nothing this experiment can resolve. Two deployment failures are reported because a simulator cannot produce them. The phone overheated on a windscreen mount in sunlight and the platform stalled the pipeline. A ninety-degree camera rotation sent detection to a null during the drive.

I. INTRODUCTION

A free-flow road network can jam without a crash and without a fixed bottleneck. A small burst of inflow pushes local density past a critical value, flow on the affected links falls, and the queue then feeds itself until the demand drains away [1], [2]. The fundamental diagram states the same fact as a curve: flow rises with density to a critical point and falls after it [3], [4], [5]. A controller that holds density below that point therefore protects throughput, and it does not need to know anything about the rest of the network to do so.

Earlier work introducing self-regulating cars built such a controller and measured it. We divided the Mainz highway network into twelve super-segments, learned a policy that assigns each super-segment a desired speed, and applied that speed to the vehicles on it. In PTV Vissim the policy raised throughput by 5% and cut average delay by 13% against no control [6]. The result holds at partial adoption and improves as the controlled fraction grows. Its deployment model is centralized. A central coordinator collects the network state,

computes the action, and sends every participating vehicle its desired speed.

This work is not the first to propose a vehicle-side traffic controller. Learned controllers that smooth mixed-autonomy traffic have been studied for years, and they report gains at low penetration in microscopic simulators [7], [8], [9]. Deployment, on the other hand, is generally treated as an engineering detail rather than as a research question, and what it would cost goes unreported, except for a few exceptions [10], [11].

This paper tackles deploying self-regulating cars as its central question. Two properties of the centralized model make its deployment costly. The first is the observation. The policy reads six features per super-segment, and four of them come from vehicle-level ground truth that a vehicle cannot easily obtain in reality. The second property is the cloud coordinator. It has to be built, paid for, kept available, and trusted, and it sits inside the control loop.

Ease of adoption is decided by the size of the change. The controller’s benefit grows with the fraction of vehicles carrying it. This fraction is therefore the quantity a deployment design has to maximise. Anything that raises the cost of one installation lowers it. A purpose-built vehicle [11], a roadside sensor [12], [13] and a per-vehicle service contract [6] each put a floor under that cost, and each has been part of a previous design. The target here is the opposite. It is the least that has to be added to a vehicle that already exists. We call this target the minimum viable deployment, and discuss it in Section II. The answer has four parts: an ordinary Android phone, one single-board computer, a traffic API, and a filter that bounds the advisory before it is displayed. The computer is an NVIDIA Jetson Orin Nano developer kit, which lists at about US\$250 [14]. The phone used for the drives reported here is a OnePlus Nord N10 (2021 model), which is a mid-range handset rather than a flagship. Neither part is purpose-built, nothing is installed on the road, and no server has to be hosted.

The division of labour inside the rig is as follows: the phone captures and forwards, the Jetson interprets and decides. Nothing is wired to the throttle, the brakes, the steering or the vehicle bus. The only connection to the vehicle is a 12 V power lead, so the installation changes no system the vehicle already depends on.

Scope of the work. No advisory this project computed has ever been acted on. On every drive, the human drove without interfere while the Jetson rendered a recommended speed on the phone’s display. There are two independent reasons for this. An experiment in which an advisory influences a human driver requires approval from an institutional review

¹Anonymous Affiliations. `anonymous.emails`

TABLE I

THE TWO LAYERS, THE SENSOR EACH READS, AND THE DECISION EACH MAKES. NEITHER SENSOR CAN SUPPLY THE OTHER LAYER’S INPUTS.

Layer	Sensor	Produces	Decides
Advisory	Traffic API, per super-segment	Aggregate speed, free flow, jam factor	The desired speed
Safety	Camera, GPS, IMU	Leader distance, closing speed, ego state	Whether to allow it

board, which this project does not hold. One instrumented vehicle also cannot produce a flow effect, so a compliance experiment on a single car would establish nothing about throughput or delay. In this paper “deployed” means that the system ran on a road and computed its outputs there. It does not mean that the recommendations were acted on. The flow-level claim is made in simulation for the same reason.

Contributions.

- A vehicle-side realization of the self-regulating-cars controller, built from commodity parts, with a sensing scheduler that sets the sampling rate of every sensor at run time rather than fixing it at install time (Sections II and III).
- A measured account of that system running on a road: eight drives with a total of 152.8 minutes on vehicle power, with per-stage timing and field provenance recorded (Section IV).
- The experiences that deployment produced, reported in the same section. The phone overheated on the road, from its own load and from sunlight on the windscreen, and a ninety-degree camera rotation sent perception to offline processing for the whole campaign.
- A flow-level measurement of the observation change. On a SUMO port of the same Mainz network, a policy restricted to the five fields a traffic API returns gains 230 ± 44 veh/h over no control, and the original six-feature policy gains 224 ± 61 veh/h (Section V).

II. THE MINIMUM VIABLE DEPLOYMENT

This section states what at a minimum has to be added to a vehicle for deploying the self-regulating cars vision.

Two sensing layers, and two observation spaces. The deployment carries two decisions, and each reads its own sensor. The advisory layer decides a desired speed for the super-segment the vehicle is on. The safety layer decides whether that speed may be shown to the driver. Table I states the split, and the two paragraphs after it state what each layer observes.

The advisory observation space. The advisory layer reads the whole network at once, as a fixed partition into super-segments, with five fields for each. Three fields come from the traffic feed, which in this deployment is the HERE Traffic API [15], and two from the map. *speed* is the segment’s current aggregate speed, which is the quantity being regulated. *free_flow* is the speed the segment carries when it is uncongested, so the two together state how far below

its own capacity the segment is running. *jam_factor* is a congestion index on a 0 to 10 scale, which summarises the same shortfall on a bounded scale the feed publishes directly. *lanes* turns a per-segment reading into a per-lane one, and *length_km* says how much road that reading describes. The feed publishes confidence and traversability as well, but neither is read.

The action is one of three speed fractions per super-segment. The base it multiplies is that segment’s static speed limit from the map, and it is deliberately not the feed’s free-flow reading, because a live reading used for the base as well as for the observation would enter the loop twice. The driver sees one number, which is the row for the super-segment the vehicle’s own position currently falls on.

The safety observation space. The safety layer reads the road immediately around the vehicle. The forward camera supplies the distance to the vehicle ahead and the rate at which that distance is closing. GPS supplies ego speed, and the slope of that speed over a short window supplies ego acceleration. The tracked vehicles in range supply a local density and a local mean speed. Those are the quantities needed to decide that a recommended speed is unsafe or obstructive at this moment, which is a judgement about the road inside the camera’s range rather than about the network.

The reasoning for the split. A traffic API reports aggregate quantities per road segment and contains no individual vehicles, so it cannot report a leader distance, a follower gap or a lane distribution. A forward camera is the opposite. It measures the vehicle ahead and reports nothing about the road beyond its own horizon. Neither sensor can do the other’s job, hence a clear split design.

Why not aggregate from local sensing. Another design could be local sensing, where a vehicle reconstructs segment-level state from its own sensors and those of its neighbours. That needs either vehicle-to-vehicle exchange, or a server that aggregates. The second option is exactly the deployment cost we want to avoid. Querying a traffic API per vehicle is simpler to operate than having vehicles talk to each other. Also, for local sensing to create a rich segment state, a sufficient number of compliant vehicles is required. Such a system can not be tested with a unit deployment. In the design reported here the per-vehicle quantities stay where they are measurable, and they feed the safety layer only.

Decentralized execution. The policy’s input is the whole network’s state, and the policy is a deterministic argmax over three speed fractions. Every vehicle that holds the same weights, reads the same map and queries the same feed computes the same action for the same super-segment. Decentralized execution therefore reproduces centralized execution by construction. What does differ on a road against the simulation is staleness. A response carries the time it was received, which this system measures. It does not carry the time at which the conditions it describes held, and nothing in the response reports that. The query is small. $O(10)$ super-segments at five features is $O(100)$ numbers per decision, and the decision interval is 60 s. The feed is queried by corridor or by circle, so one request covers the whole extent and the

number of super-segments does not change the number of requests.

What the vehicle must therefore carry. Five things. (1) A network definition that names the super-segments and their edges. (2) A traffic-feed query bounded by that network’s extent. (3) A policy trained on that network. (4) A local sensor set for the safety layer. (5) A display. There is no connection to the throttle, the brakes, the steering or the vehicle bus, and no server inside the control loop.

III. THE SYSTEM AS BUILT

The rig. The vehicle side is an ordinary Android phone and a Jetson Orin Nano, with no purpose-built hardware (Figure 1). The phone holds no interpretation logic. It captures or queries at the rate it is told and forwards raw data, and every association, fusion and sampling decision happens on the Jetson. The phone is connected to the internet via 5G cellular connection. Two transports are implemented for the same wire protocol. A Tailscale network is used for development with the phone in hand, and USB is used in the vehicle. Both devices are powered from the vehicle, so energy is not a binding constraint.

The sensing scheduler. Each sensor’s sampling rate is set at run time by the Jetson and commanded to the phone. The four rates are camera, GPS, inertial unit and traffic feed. The costs the scheduler trades against are traffic-API quota, thermal headroom on the phone, and Jetson compute. It has two modes. In shadow mode the decision is recorded and not sent, and in live mode the phone applies it. That distinction is about sensing configuration only. The rate command carries sampling rates and a shadow flag. The word “live” here describes what the sensing configuration did, and not what the vehicle did.

The transport. One wire protocol carries everything between the phone and the Jetson, over the development link and the in-vehicle link alike. It defines eight channels, each with its own sequence numbers, priority, overflow policy and queue depth, over a shared timebase. The frame layout is frozen in a single specification file. Neither implementation is tested against the other. Both are tested against that file, so the two sides cannot drift by agreeing with each other.

The safety gate. The advisory passes a filter before it is displayed. The filter is vendored onto the device from the simulator’s safety layer and checked against a frozen copy of it, so the two cannot drift apart. The simulator does not run the layer itself, and Section V gives the reason. The advisory is a speed, so a rule earns its place by bounding that speed, and two do. The first raises a recommendation that would have the vehicle travel slowly on an uncongested road, which is the obstruction case. The second overrides the recommendation when the gap to the vehicle ahead, or the time left to close it, falls below the configured minimum. Each rule records, per tick, whether it fired, stayed quiet, or could not be evaluated at all, with the missing inputs named. A rule that cannot be evaluated is inert, so it cannot move the recommendation in either direction.

TABLE II

THE WHOLE ROAD CORPUS. ONE DAY, ONE VEHICLE, EIGHT RUNS.

runs	8: six shadow (17,948 ticks), two live (4,981 ticks)
ticks	22,929, GPS fix valid on 22,734 (99.1%)
recorded span	152.8 min, of which 114.3 min above walking pace
distance	between 86.55 and 90.06 km
video	1.676 GB, 19,000 indexed frames across six runs
traffic feed	123 response bodies, 3.7 MB, every one HTTP 200
inertial archive	658,862 raw samples in 783,519 records 187 files, 2,180,405,759 bytes, verified by digest against the device

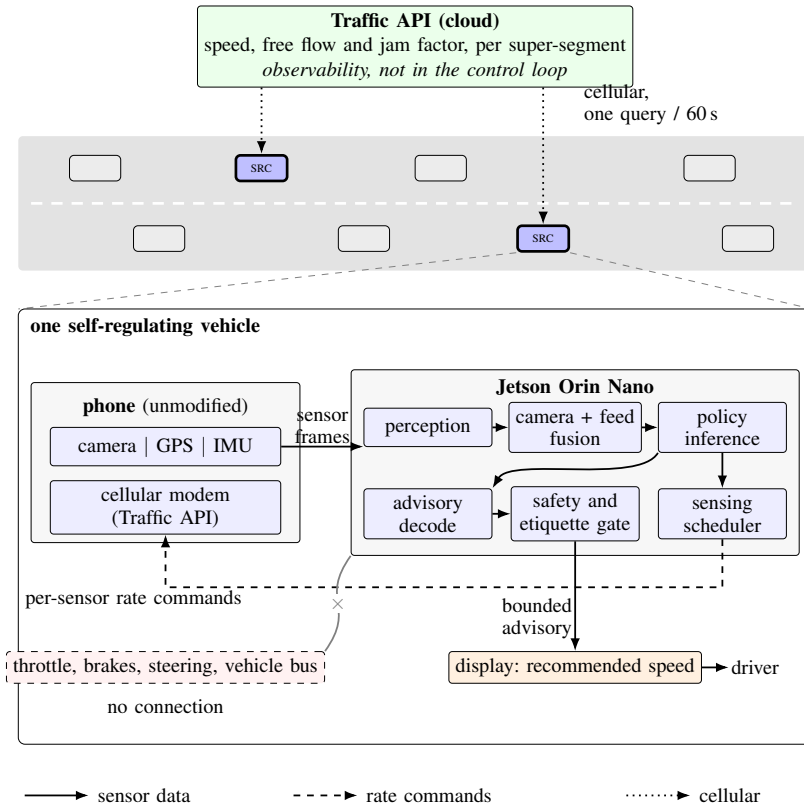
The device cannot drift from what was trained. The observation and action contract is vendored onto the device from the simulator and pinned to a named version of it, so the device encodes an observation and decodes an action exactly as training did. The export step refuses any dimension mismatch. We enforce this using a second frozen specification file, holding encoded observations, the action schema, the decoders and the neutral fallbacks. Every entry in it is derived twice, once from the pinned version of the simulator and once from the vendored copy, and the generator refuses to write the file on any disagreement. That derivation runs where the training stack lives. The check that runs on the device is the comparison against the frozen file, which needs the file and nothing else, so the property is testable on the hardware that has to hold it.

Two design stances. The first is to degrade rather than stop. Missing GPS, no camera, no display and no trained checkpoint each have a defined degraded mode, because debugging time inside a vehicle is expensive. The second is that the hot path holds no queues, and the latest value wins. A frame that was never processed is dropped, a slow display skips ticks, and a stalled card drops log records. For a real-time advisory, stale data is worse than missing data.

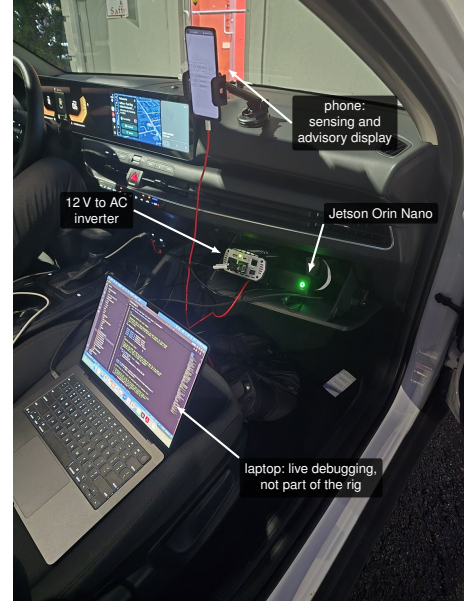
Data collection. A tick is one pass of the Jetson’s pipeline. It takes the most recent camera frame together with the latest GPS and inertial readings, runs detection, tracking and distance estimation, builds the observation, runs the policy, decodes the advisory, passes it through the safety gate, and writes one record. Camera frames pace the ticks, so the tick rate follows the camera rate the sensing scheduler has commanded. Every per-tick count in this paper refers to that record.

IV. RUNNING IT ON A ROAD

The system was driven on 2026-09-08 in Westfield, New Jersey, on a OnePlus Nord N10. Table II is the whole corpus. Distance is reported as a bracket rather than a number. Summing great-circle steps between fixes less than 3 s apart gives 86.55 km and omits distance covered during gaps. Integrating the reported speed gives 90.06 km and counts the gaps but trusts the speed field. Quoting either figure alone



(a) The deployed architecture.



(b) The rig installed in the vehicle.

Fig. 1. The deployed system. In (a) ordinary traffic is grey and self-regulating vehicles are blue. Each self-regulating vehicle queries the feed on its own, so there is no vehicle-to-vehicle link and no coordinator. Inside one vehicle, the phone captures or queries at the rate it is told and forwards raw data, and every interpretation and sampling decision is on the Jetson. In (b) the same parts installed in the vehicle. The inverter supplies mains power from the vehicle’s 12 V outlet, and the laptop was attached for live debugging and is not part of the deployed rig.

would overstate the precision.

The shakedown drive. The first drive ran 1,632 ticks over 279.5 s and 6.24 km, at a mean speed of 22.9 m/s. The mean tick rate was 4.69 Hz with a median spacing of 203 ms, and two gaps longer than 2 s totalled 5 s. Distance agrees to 0.01 km between integrating the reported speed and the great-circle path through the fixes, which is two independent routes to one number.

The drive also produced three defects. The camera frames arrived rotated by ninety degrees. The frames were being discarded, so the drive could not explain itself. The severe thermal tier ended a session rather than lowering rates. These were fixed live using the laptop as shown in Figure 1.

Latency. Table III gives the two measurements. The on-Jetson path is a bench measurement on pre-stamped frames, so it excludes capture wait. The phone-to-Jetson figure is the pooled 95th percentile over three bench runs of 180 s each. Over USB it is 116.19 ms against a 200 ms target, and over the Tailscale network it is 215.63 ms. The largest and smallest of the three USB runs differ by 48.80 ms at the 95th percentile, and that difference has one primary source. The phone-side send-queue stage differs between the same pair by 46.40 ms, while every other stage differs by less than 2 ms. The spread is phone-side queueing rather than the link. The wire hop is reported as an order rather than a duration

on purpose. Its two endpoints are stamped on different devices, and therefore on different clocks. Measuring it means converting between them, and every such conversion carries an error bound of about half the minimum round-trip time. Its median is 1.97, 2.16 and 2.03 ms across the three runs, against conversion bounds of 1.95, 1.86 and 2.06 ms for the same runs. The quantity is on the same order of the error bar on the clock conversion used to measure it, and hence is not separable from that bound.

Every drive reproduces its own recorded decisions. A replay tool re-runs each logged tick’s inputs through the incumbent sensing scheduler and requires the recorded decision back. All eight drives pass: 0 of 22,929 ticks mismatched, with the success exit code on every run. The claim these runs support is that each log reproduces the decisions it recorded, and not that the logs are uncorrupted.

The two clocks, and conversion between them. The phone and the Jetson each run their own clock, and the two are not synchronised. Any interval with one end on each device has to be converted, and a conversion can be wrong. So no conversion here returns a bare number. It returns the instant, how far that instant could be off, and which estimate produced it. An estimate too poor to be useful makes the conversion fail, rather than return an answer with a margin wide enough to be useless. The margin is half the fastest

TABLE III

MEASURED LATENCY. THE UPPER BLOCK IS THE ON-JETSON COMPUTE PATH, MEASURED ON THE DEVICE ON PRE-STAMPED FRAMES. THE LOWER BLOCK IS THE PHONE-TO-JETSON PATH, POOLED OVER THREE 180 S BENCH RUNS, 2,684 TICKS.

Stage	Measured
detection (YOLOv8n, TensorRT FP16)	17.7 ms p50, of which 3.9 ms is GPU
tracking and distance	1.1 ms p50
observation and encode	0.4 ms p50
actor and advisory decode	0.5 ms p50
on-Jetson end to end	19.8 ms p50, 20.2 ms p95, 48.5 FPS
phone to Jetson over USB	116.19 ms p95 (target 200 ms)
the same path over Tailscale	215.63 ms p95
the USB wire hop itself	of the order of its own 2 ms conversion bound

round trip the two devices have exchanged, plus whatever they have drifted since. A round trip cannot reveal how its time divided between the outbound and the return leg, so half of it is the part that cannot be reasoned away.

Three measurements bound that machinery. The first is a null case, with both ends of the exchange on one machine, so the true offset is zero by construction. The estimate stayed within 12 microseconds over 145 s and fitted a drift of -0.05 ppm, so it invents no structure where there is none. The second is a real link for 330 s: 357 exchanges, and nine outcome counters that account for every ping sent. The fastest round trip had a median of 14.8 ms and the margin a median of 8.0 ms. The third is the same link under full sensor load, a 10 Hz camera at 40 KB per frame and a 50 Hz inertial unit, where 218 exchanges completed with every channel still on cadence. Across these runs the two clocks drift apart at 12 parts per million, against the 50 the design budgets for.

Deployment experiences. The rest of this section reports the challenges we faced during our unit deployment. They are not incidental to the contribution, because they are the part of it that cannot be obtained from a simulator.

Thermal behaviour is the binding constraint. The thermal backoff works on hardware. Across two 900 s bench runs the thermal rule fired on 1,504 of 3,486 ticks and on 1,561 of 3,451 ticks. The recorded cause was a warm skin temperature. The phone delivered 3.000 and 2.998 Hz against a commanded 3.0 Hz.

On the first drive, the handset reached the severe thermal tier on a windscreen mount in sunlight, with one zone at 51.6 C. The backoff behaved correctly and commanded 1.5 Hz. What ended the drive was the platform throttling the application and stalling the pipeline. The final tick recorded a link time of 135,359 ms against a median of 52 ms, and the application then closed the session. Nothing in this system detects or reports that condition, and no gate reads the link time. The failure was sharp rather than gradual. Every tick up to 1311 had a normal link time, so 279 s and 6.24 km of that drive remain usable and only the last tick does not.

TABLE IV

THE SAME TWO FRAMES THROUGH THE SAME DETECTOR, WITH ONE ROTATION APPLIED. THE DETECTOR IS UNCHANGED BETWEEN THE TWO COLUMNS.

As delivered	Rotated 90° clockwise
0 vehicles	2 vehicles, confidence 0.90
0 vehicles	5 vehicles, confidence 0.87

Perception fault. The camera frames arrive rotated by ninety degrees. The detector expects an upright road scene, so on a rotated frame it returns almost nothing. Table IV is the measurement that identified the cause. The same frames through the same detector give nothing as delivered and several vehicles once rotated clockwise, and only the clockwise rotation recovers them. The frame shows a van filling much of the picture with a legible plate, so this was not a marginal detection.

The vehicle is a hostile environment and the faults are physical. Both devices ran from vehicle power for 152.8 minutes across eight runs with no power interruption. Two install faults each cost drive time. Cable type is not interchangeable, because one handset’s cable failed in the vehicle, and the application was moved to a second handset mid-session with the swap recorded automatically in the run metadata. Direct sunlight on the dashboard heated the tethering handset enough that its hotspot shut off, which ended a drive. The mount needs shade and not only a clamp.

V. THE FLOW-LEVEL RESULT

One instrumented vehicle cannot demonstrate throughput or delay. The flow-level claim therefore comes from simulation. The question the simulation answers is narrow. It is whether restricting the controller’s observation to the fields a traffic API returns costs throughput.

The two observations being compared. The prior work controller [6] reads six quantities per super-segment: density, lane count, mean speed, mean gap, entry rate and exit rate. Four of the six are vehicle-level ground truth in the published implementation, and a vehicle cannot obtain them. The gap is the mean of each vehicle’s follow distance. The entry and exit rates are set differences over the vehicle identities present one minute apart. The density is counted over the vehicles on the segment. The deployed observation of Section II replaces those six with the five fields a feed and a map supply. The reward is not changed by the substitution. Training is centralized, so it may use privileged information, and it keeps the published form on ground-truth density. Only the state the controller reads at decision time differs between the two arms.

On the road those five fields come from the HERE Traffic API [15], whose *jam factor* is a congestion index on a 0 to 10 scale. The simulator has no such feed. Speed and free flow are read from the road itself, and the jam factor is computed as $\text{clip}(10(1 - \text{speed}/\text{freeFlow}), 0, 10)$. HERE does not publish the formula behind its own jam factor,

and it folds in incident data this project does not have. That expression is therefore a stated model rather than a measurement, and it is the one place where the simulated observation and the deployed observation are not the same quantity.

The port. The published controller [6] ran on the Mainz network in PTV Vissim under a Wiedemann-99 car-following model. That network was converted to SUMO [16] directly from the Vissim XML. The conversion produces 410 edges over 51.5 km. Three conversion decisions matter. The source network is flat in the dynamics Vissim runs, because every one of its 410 links carries a gradient of zero. Its geometry nevertheless carries vertical offsets of up to six metres, which separate overpasses from the roads beneath them rather than describing terrain. The importer reads gradients off that geometry and produces edges far steeper than any road, up to 923%, and SUMO applies gradient to acceleration. Flattening the import therefore restores the source’s own dynamics. Every edge imports with a 50 km/h limit while the action set tops out at 60 km/h, so a speed factor of 1.2 lifts the limit to 60 km/h and all three actions become expressible. The exit is modelled as a three-into-two lane drop, because the source link ends into free outflow and the network otherwise has no bottleneck at that point. The conversion also does not import right-of-way: the port carries 2 junctions with conflicting movements against 10 active conflict areas in the source. All three decisions apply identically to every experiment.

The fleet, and why it is not the published one. The fleet is the Extended Intelligent Driver Model with a one-second reaction time [17], [18]. It was chosen because it has a capacity drop. A capacity drop is not a free parameter of a simulation. Real queue discharge runs 5 to 20% below free-flow capacity, which is a property of traffic rather than of a model [19]. Measured on this port, the Wiedemann-99 calibration discharges 4.3% faster than free flow, which is inconsistent with real world measurements. Its queues discharge at 1,980 veh/h against 1,899 veh/h free-flowing. The Extended Intelligent Driver Model with a one-second reaction drops 21% at an isolated lane drop. A simulation that fails to reproduce a measured phenomenon is a settings problem, not evidence that the simulation is the reference. The gate applied before any experiment was run is that served flow must fall as offered demand rises, by more than the seed spread.

Protocol. Seeds 1 to 10 train, seeds 11 to 15 select the checkpoint, and seeds 16 to 30 evaluate. Four arms across fifteen seeds is 60 episodes. Each episode runs 2,500 s at a 0.5 s step, with the measurement window opening at 900 s, under a sustained demand of 4,500 veh/h and 3,124 scheduled departures, at 100% penetration.

Why the window opens at 900 s. Served flow is zero until the first trips complete, and it does not reach its sustained level before then. Over 200 s windows, averaged across the fifteen evaluation seeds, the no-control arm completes 0, 43, 79 and 139 trips in the four windows before 800 s, and between 188 and 214 in every window after. Measuring throughput from the start of the episode would average

TABLE V

HELD-OUT SEEDS 16 TO 30 ON THE MAINZ PORT, PAIRED ON SEED. THE ZEROED-HEAD ARM IS THE SAME NETWORK WITH ITS OUTPUT LAYER ZEROED, AND IT CARRIES THE ENTRY GATE THAT BOTH CONTROLLED ARMS CARRY.

Arm	Flow (veh/h)	Paired gain	Speed (km/h)
Traffic-API observation	3,765	$+230 \pm 44$	40.7
Published six features	3,759	$+224 \pm 61$	41.0
No control	3,535	reference	42.3
Zeroed output layer	2,847	-688 ± 46	28.8

that fill into the result. The network itself does not settle. Insertions exceed arrivals in every window of the episode, so occupancy rises from 776 vehicles at the window start to 1,141 at 2,400 s, which is the last sample. The window makes the throughput figure a measurement of the controller rather than of the fill, and it does not make the episode a steady state. Arms are paired on seed, because each arm runs the same traffic realization, and every error bar below is two standard errors of the paired difference.

The result. Table V gives it. Restricting the observation to what a traffic API returns costs nothing that this experiment can resolve. The two controlled arms differ by 6 veh/h against error bars of 44 and 61 veh/h, which means that performance of the deployed version is inseparable from prior work. This shows that our model of decentralized execution does not lose against the centralized controller.

Both controlled arms run slower than no control while serving more vehicles. Space-mean speed falls from 42.3 to 40.7 km/h and arrivals rise from 1,940 to 1,995 per episode. That is the intended mechanism. Holding density below critical upstream of the merge lets the merge discharge near capacity instead of breaking down.

Testing the boundary. Both controlled arms hold a vehicle at an entry link while that link is at or above critical density. The no-control arm does not. This gate is a boundary condition on the controlled system rather than a piece of road. Applying it to an uncontrolled run would credit that run with a control action it is not taking. That asymmetry is a candidate explanation for the gain, so it was tested. The zeroed-head arm is the same trained network with its output layer zeroed after loading. It carries the entry gate, and it loses 688 ± 46 veh/h against no control, which is 15.7 times the no-control arm’s own bar below it. The gate alone does not produce the gain.

What this result is not comparable to. No number here is comparable to the published table in [6]. Every claim is against this port’s own no-control baseline. Table VI states what the two papers cover between them. The controller and the shape of the topology are what the two runs have in common. Almost everything that decides how the controller behaves differs, which is why the pair is stronger evidence than a matched number in one simulator would be. A topology or demand sweep here would re-establish what the earlier paper already did.

The simulated gate and the deployed gate answer dif-

TABLE VI

WHAT THE TWO PAPERS COVER BETWEEN THEM. THE ROAD NETWORK IS NOMINALLY THE SAME, AND THE PORT DIFFERS FROM THE SOURCE IN RIGHT-OF-WAY, FLEET AND EXIT GEOMETRY.

	Earlier paper	This paper
simulator	PTV Vissim	SUMO
car following	Wiedemann-99	EIDM, 1 s reaction
demand	18,000 veh/h surge	4,500 veh/h sustained
network	Mainz	Mainz

ferent questions. In simulation the advisory is written as a speed command, and SUMO bounds it by the car-following model’s own safe speed. A vehicle whose leader is slower follows its leader. That is the same semantics the published implementation has, and it is the documented way to command a speed in SUMO. We left it alone. Replacing the simulator’s car following with our own layer would make this arm incomparable to the earlier paper, whose own car-following model is the row that differs in Table VI. It would also make the run reproducible only by someone holding our code, rather than by someone holding SUMO and the network file.

On the road that bound would be the wrong instrument. SUMO’s safe speed is an actuator command, and it works because the simulator sets the vehicle’s speed. The prototype sets nothing. The driver does the car following, so the vehicle already carries the model the simulator supplies, and a second copy of it on the phone would recompute what the driver is already doing. The question the road poses is whether a recommendation is fit to show, and that is what the deployed filter decides. One rule raises a recommendation that would have the vehicle travel slowly on an uncongested road. The other withholds the number when the gap to the vehicle ahead, or the time left to close it, falls below the configured minimum. Neither has an analogue in the simulator, because a simulated vehicle has no display and no driver reading it.

VI. RELATED WORK

The controller this paper deploys. Self-regulating cars learns a policy that assigns a desired speed to each super-segment of a highway network, and reports a 5% throughput gain and a 13% delay reduction against no control in PTV Vissim [6]. Its observer is central and its decisions reach vehicles through a cloud service. The failure mode it targets is characterized in [1], which defines a sudden traffic jam as a burst that pushes density past a critical threshold into a stable low-capacity state. This paper changes where the policy is evaluated and what it is allowed to read, and it leaves the reward, the training and the controller unchanged.

Sparse vehicles as traffic actuators. A single controlled vehicle damps stop-and-go waves on a ring track [10], which established the sparse-actuator premise. Learned controllers improve simulated traffic at low penetration in the mixed-autonomy literature [7], [8], [9]. CIRCLES placed about one hundred controlled vehicles into live interstate traffic and

is the strongest field evidence available, using purpose-built vehicles and a centralized speed planner carried over cellular links [11]. The work here differs on the deployment axis rather than the control axis. The vehicle is unmodified, the compute is commodity, and no planner is contacted at run time.

Infrastructure-centric control. Ramp metering, variable speed limits and adaptive signal control remain the deployed tools [12], [13], [20], [21]. They are effective where the sensors and actuators exist, and they require owning the roadside.

Traffic state from vehicles and from feeds. Probe-vehicle systems estimate network state by uplinking per-vehicle traces to a backend that aggregates centrally [22], [23]. Connected-vehicle standards broadcast per-vehicle position, speed and heading at up to ten hertz [24], [25], [26], and the collective perception service extends that to per-object detections [27]. This design uses neither. The observation it needs is already published as a commercial product at segment granularity [15], so a query replaces both the radio and the aggregation backend. The trade is explicit. A feed is a third-party dependency with an unreported lag, and carries different risks, but is suitable to achieve a minimum viable deployment.

Advisory interfaces. Coarse, human-followable advisories have precedent in advisory-autonomy work [28], [29], [30], and mobile telematics platforms operate the driver-facing rail at the scale of tens of millions of drivers [31], [32]. This paper does not evaluate compliance, for the reasons stated in Section I.

Edge perception. Embedded detectors and Jetson-class hardware make on-vehicle perception practical [33], [34], [35], [36]. The measurements in Section IV are consistent with that literature. The contribution here is not the detector. It is what the rest of the system around it has to do to keep the detector’s output honest when it is wrong.

VII. CONCLUSION AND LIMITATIONS

Moving this controller onto a vehicle is a systems problem rather than a control problem. The parts are a phone the driver already owns, a US\$250 single-board computer, a traffic API, and a filter on the advisory. Nothing is installed on the road, and no server sits inside the control loop. The compute device’s median is 19.8 ms and the link’s 95th percentile is 116.19 ms against a 200 ms target. Restricting the observation to what a feed returns changes throughput by 6 veh/h against error bars of 44 and 61 veh/h, while both controlled arms gain more than 200 veh/h over no control. What decided whether a drive produced data was thermal, physical and perceptual rather than any of this.

What the deployment does not establish. No advisory was acted on, so nothing here measures compliance. The corpus is eight runs on one day, with one vehicle, one driver and one road. The frame rotation left the detector returning almost nothing, so the safety layer’s camera inputs were processed offline. The wire hop is an order rather than a duration, because its median and its conversion bound are

the same size. The feed reports when a response arrived and not when the conditions held, so nothing bounds the observation's staleness. The first drive ended in a condition nothing detects. Distance is a bracket rather than a number, because the two methods disagree by 3.5 km.

What the simulation does not establish. Every figure is against this port's own baseline, and none is comparable to the published table. The port differs from its source in right-of-way, exit geometry and gradient, and the fleet is not the published one. The episode does not reach a steady state. The jam factor the simulated observation reads is a stated model rather than a measurement.

VIII. ACKNOWLEDGEMENTS

Opus 5 (1M) model was used for writing the code as well as the text of the paper. All sections in the paper and code was co-worked on by AI and human authors, with human verification.

REFERENCES

- [1] A. Authors, "Anonymous title," *Anonymous Publication*, Anonymous Year.
- [2] Y. Sugiyama, M. Fukui, M. Kikuchi, K. Hasebe, A. Nakayama, K. Nishinari, S.-i. Tadaki, and S. Yukawa, "Traffic jams without bottlenecks: experimental evidence for the physical mechanism of the formation of a jam," *New journal of physics*, vol. 10, no. 3, p. 033001, 2008.
- [3] B. D. Greenshields, J. R. Bibbins, W. Channing, and H. H. Miller, "A study of traffic capacity," in *Highway research board proceedings*, vol. 14. Washington, DC, 1935, pp. 448–477.
- [4] M. J. Lighthill and G. B. Whitham, "On kinematic waves ii. a theory of traffic flow on long crowded roads," *Proceedings of the royal society of london. series a. mathematical and physical sciences*, vol. 229, no. 1178, pp. 317–345, 1955.
- [5] P. I. Richards, "Shock waves on the highway," *Operations research*, vol. 4, no. 1, pp. 42–51, 1956.
- [6] A. Authors, "Anonymous title," in *Anonymous Publication*, Anonymous Year.
- [7] C. Wu, A. Kreidieh, K. Parvate, E. Vinitzky, and A. M. Bayen, "Flow: A modular learning framework for mixed autonomy traffic," *arXiv preprint arXiv:1710.05465*, 2017.
- [8] E. Vinitzky, A. Kreidieh, L. L. Flem, N. Kheterpal, K. Jang, F. Wu, C. Wu, R. Liaw, E. Liang, and A. M. Bayen, "Benchmarks for reinforcement learning in mixed-autonomy traffic," in *Proceedings of the 2nd Conference on Robot Learning*. PMLR, 2018, pp. 399–409.
- [9] A. R. Kreidieh, C. Wu, and A. M. Bayen, "Dissipating stop-and-go waves in closed and open networks via deep reinforcement learning," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 1475–1480.
- [10] R. E. Stern, S. Cui, M. L. Delle Monache, R. Bhadani, M. Bunting, M. Churchill, N. Hamilton, R. Haulcy, H. Pohlmann, F. Wu, B. Piccoli, B. Seibold, J. Sprinkle, and D. B. Work, "Dissipation of stop-and-go waves via control of autonomous vehicles: Field experiments," *Transportation Research Part C: Emerging Technologies*, vol. 89, pp. 205–221, 2018.
- [11] CIRCLES Consortium, "Traffic control via connected and automated vehicles: An open-road field experiment with 100 cavs," *arXiv preprint arXiv:2402.17043*, 2024.
- [12] E. D. Arnold, Jr., "Ramp metering: a review of the literature," Virginia Transportation Research Council, Tech. Rep., 1998.
- [13] M. Papageorgiou and A. Kotsialos, "Freeway ramp metering: An overview," *IEEE transactions on intelligent transportation systems*, vol. 3, no. 4, pp. 271–281, 2002.
- [14] NVIDIA Corporation, "NVIDIA jetson orin nano developer kit," <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>, 2025, accessed: 2026-09-12.
- [15] HERE Technologies, "HERE traffic API v7 developer guide," <https://www.here.com/docs/bundle/traffic-api-v7-api-reference>, 2025, accessed: 2026-09-12.
- [16] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using SUMO," in *Proceedings of the IEEE Intelligent Transportation Systems Conference (ITSC)*, 2018, pp. 2575–2582.
- [17] D. Salles, S. Kaufmann, and H.-C. Reuss, "Extending the intelligent driver model in SUMO and verifying the drive off trajectories with aerial measurements," in *SUMO Conference Proceedings*, vol. 1, 2020, pp. 1–25.
- [18] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.
- [19] F. L. Hall and K. Agyemang-Duah, "Freeway capacity drop and the definition of capacity," *Transportation Research Record*, no. 1320, pp. 91–98, 1991.
- [20] Transport for New South Wales, *SCATS Traffic System: Sydney Co-ordinated Adaptive Traffic System*, <https://scats.transport.nsw.gov.au/>, 2024, accessed: 2024-10-21.
- [21] Transport Research Laboratory (TRL), *SCOOT: Split Cycle Offset Optimisation Technique*, <https://trl.co.uk/solutions/scoot>, 2024, accessed: 2024-10-21.
- [22] T. Seo, A. M. Bayen, T. Kusakabe, and Y. Asakura, "Traffic state estimation on highway: A comprehensive survey," *Annual Reviews in Control*, vol. 43, pp. 128–151, 2017.
- [23] J. C. Herrera, D. B. Work, R. Herring, X. Ban, Q. Jacobson, and A. M. Bayen, "Evaluation of traffic data obtained via GPS-enabled mobile phones: The Mobile Century field experiment," *Transportation Research Part C: Emerging Technologies*, vol. 18, no. 4, pp. 568–583, 2010.
- [24] SAE International, "V2X communications message set dictionary," SAE International, Tech. Rep. J2735, 2020.
- [25] ETSI, "Intelligent transport systems (ITS); vehicular communications; basic set of applications; part 2: Specification of cooperative awareness basic service," European Telecommunications Standards Institute, Tech. Rep. EN 302 637-2 V1.4.1, 2019.
- [26] J. B. Kenney, "Dedicated short-range communications (DSRC) standards in the united states," *Proceedings of the IEEE*, vol. 99, no. 7, pp. 1162–1182, 2011.
- [27] ETSI, "Intelligent transport systems (ITS); vehicular communications; basic set of applications; collective perception service; release 2," European Telecommunications Standards Institute, Tech. Rep. TS 103 324 V2.1.1, 2023.
- [28] Z. Yan, A. R. Kreidieh, E. Vinitzky, A. M. Bayen, and C. Wu, "Unified automatic control of vehicular systems with reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 2, p. 789–804, Apr. 2023. [Online]. Available: <http://dx.doi.org/10.1109/TASE.2022.3168621>
- [29] J.-H. Cho, S. Li, J. Kim, and C. Wu, "Temporal transfer learning for traffic optimization with coarse-grained advisory autonomy," *arXiv preprint arXiv:2312.09436*, 2023.
- [30] L. Fridman, "Human-centered autonomous vehicle systems: Principles of effective shared autonomy," *arXiv preprint arXiv:1810.01835*, 2018.
- [31] Cambridge Mobile Telematics, "Cambridge mobile telematics," 2025, accessed: 2025-05-15. [Online]. Available: <https://www.cmtelematics.com/>
- [32] Samsara Inc., "Samsara: The connected operations cloud," 2025, accessed: 2025-05-15. [Online]. Available: <https://www.samsara.com/>
- [33] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [34] NVIDIA Corporation, *NVIDIA Jetson Platform Services Documentation*, 2023, accessed: 2025-08-22. [Online]. Available: <https://docs.nvidia.com/jetson/jps/index.html>
- [35] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.
- [36] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 10 371–10 381.